

Correct Optimizations: Phase 1 Report

Based on Minotaur: A SIMD-Oriented Synthesizing Superoptimizer [1]

Kilian Schulz
Technical University Munich

Ilja Gamza
Technical University Munich

Tomás Gaspar
Technical University Munich

Claudia Dünzinger
Technical University Munich

1 Introduction

This report aims to analyze and propose opportunities to continue the research from Liu et al. [1].

2 Paper Summary

2.1 Context

Minotaur is a superoptimizer that tries to identify missed optimization opportunities by the LLVM compiler by slicing generated IR and trying to synthesize new faster code. Its synthesizer creates improved machine code, generating and comparing equivalent but cheaper instruction sequences. To guarantee correctness of the optimization, Alive2 was extended to formally verify the generated code. Additionally, to reduce computation overhead, Minotaur also caches discovered transformations for reuse them in future compilations.

The use case of Minotaur is to further optimize already heavily optimized code from a compiler which still often contain suboptimal code segments, particularly for SIMD operations. The authors showed that even LLVM’s advanced autovectorizer produced redundant or inefficient SIMD instructions. For those missed opportunities, Minotaur can correct these suboptimal instruction sequences by replacing them with improved machine code during the compilation. Some of those optimizations could be generalized and added to LLVM. Hence, the authors enabled higher performing and additionally formally verified machine code generation through the use of Minotaur.

Before Minotaur, several superoptimizer already existed that search through the code, be it assembly or LLVM IR, for more efficient sequences instead of performing set rewriting rules like traditional compilers. Common examples include Souper – with focus on scalar integer optimizations and which also uses LLVM IR – and STOKE, which uses stochastic search for faster x86 code blocks. However, those tools only addressed part of the problem. Souper [2] detects only improvements for integer operations, but not for floating-point or

SIMD instructions. STOKE’s [3] improvements include complex optimizations, but the tool relies on randomized testing and not formal verifications for its transformations. Therefore, the combined challenge of performing complex vector or floating point optimizations, formally verifying the transformation and integrating those capabilities into production compilers remained unsolved, until Minotaur.

Designing a efficient and high performing superoptimizer included several challenges. First, the search space of replacement options for a suboptimal code sequence is enormous, making an exhaustive search for the optimal choice computationally expensive. Second, the semantic formal and not random equivalency check adds further overhead – especially for hard to verify SIMD and floating point operations. Minotaur relies on SMT solvers for equivalency checks between the original code and the proposed optimization, but these solvers often struggle with large complex expressions. At the same time a sufficiently large number of lines of code is needed to provide enough context to be able to find improvement options in the first place. Furthermore, since Minotaur aims for low level, hardware aware optimizations, the authors used detailed information about pipelines and cache effects to accurately model the performance of optimizations. These difficulties of realizing high performance without too much computational cost, required a multiple stages process.

2.2 Contributions

Because today’s SMT solvers cannot optimize entire LLVM functions, Minotaur instead optimizes each instruction individually. It therefore breaks the LLVM function in SSA form down into cuts, which are small loop-free subsets of the the instruction’s dependencies, to capture where the data used by the instruction came from. The size of the cut hereby influences the optimization, as too small cuts contain not enough context to determine a better code sequence, while too large cuts increase the solvers computation time dramatically. Minotaur’s cut extraction is noticeably better than previous ones since it is able to handle memory operations, vector op-

erations and floating-point operations. Through an empirical study the authors selected the cut size of four as it provides a good balance between finding most optimization while still running in an acceptable amount of time. For each cut, a set of potential alternatives which are faster are synthesized. These improvements are then verified with Alive2, which the authors extended to support SIMD operations. Lastly, for each optimization its speedup is estimated using a more accurate model (LLVM MCA) and its original and transformed code is cached for reuse in future compilations.

To measure how Minotaur’s optimizations influence the overall performance of larger software projects, the authors benchmarked Minotaur on the SPEC CPU 2017, GMP and libYUV on Intel and AMD hardware with LLVM 18 compiled with -O3 and -march=native. They determined a maximum speedup of 1 - 7 percent, where integer SIMD fragments made up 75 percent of the discovered rewrites. When the compilation is performed with an empty cache, Minotaur increased compile times significantly. With a filled cache, however, only an additional 3 percent of computation time was needed. Overall, the authors found 15 simple and general floating-point and vector optimizations, which they then reported to the LLVM project.

3 Artifact Evaluation Results

3.1 Project Issues and Benchmarking Setup

First, it has to be mentioned that the paper did not provide a exact git commit hash for the Minotaur version that was used by the authors to run their benchmarks. Since we could not determine which version was used it was decided to evaluate the most up to date version of Minotaur¹. One important difference between the most recent version and the version referenced by the authors in the paper is that the version in the paper relied on LLVM 18 whereas the newest version uses LLVM 20.

Second, the most up to date version of Minotaur, as of writing, does not work out of the box. For it to not immediately crash on startup, the order in which the libraries are linked by Minotaur had to be changed. This changes the order in which the offending dynamic libraries are loaded when the program is started and thereby fixing the issue of undefined symbols.

Third, Minotaur uses Alive2’s LLVM interpreter which is listed as highly experimental² and caused multiple crashes. Some of which where not reliably reproducible and could therefore not be debugged. In order to mitigate this problematic source files were filtered. More about that when the setup for libyuv is discussed. Other bugs could be fixed.

¹<https://github.com/KiSchulz/Minotaur/tree/8ae5395286ba40fa29c16a0229fd3d6e9795d4cd>

²<https://github.com/AliveToolkit/Alive2/tree/master?tab=readme-ov-file#running-the-standalone-llvm-execution-tool-alive-exec>

Fourth, the caching of Minotaur seems to not fully work in the last release, as compiling a source file multiple consecutive times still leads to cache misses. This might be caused differing inputs from LLVM’s side.

After those issues were addressed, as far as possible, a run could be attempted. Everything was compiled with the -O3 and -march=native flags as described in the paper. We benchmarked libgmp-6.2.1, as the paper did, and a fitting version of libyuv³, which is able to build with LLVM 20. All runs were performed with the same parameters as the paper i.e. a cutting depth bound of 4, total synthesizes time of 5 minutes and a 1 minute timeout for individual SMT queries. The benchmarks were build and run on an AMD Ryzen 9 5900X whereas the authors used an AMD Ryzen 9 5950X. The former has 12 cores running at up to 4.8 GHz whereas the latter has 16 cores running at up to 4.9 GHz.

3.2 libgmp

According to the paper the build of libgmp with an empty cache took 440 minutes. Our build took approximately 1020 minutes while using a partly dirty cache from previous attempts. It seems unlikely that the discrepancy in build times are caused by the difference in hardware. This difference in compile time might be due the authors using the offline mode of Minotaur, which we intentionally did not use. As discussed earlier the cache did not work as expected and using the offline mode would effectively have doubled the compile time. The reason for this that after creating the cuts for each cut Minotaur tries to synthesize a faster version. After that is done another run of Minotaur is required but since to apply the cached optimizations. But because this is not working properly this leads to another long compilation.

After building libgmp the provided tests were run. Using regular LLVM all tests passed without any issue. In contrast, using Minotaur at least two tests failed namely t-root and t-perfpow. This indicates some correctness bug in Alive2 or in how Minotaur uses Alive2. Later more of that is discussed.

We decided to ignore the failing tests and to benchmark libgmp. An improvement in runtime, by using Minotaur for building, could not be shown in the benchmark (1). There could be multiple reasons for this. One could be that any of the previous issues which we were not able to solve caused this. Or, alternatively that most of the relevant optimization found by Minotaur in LLVM 18 are now already present in LLVM 20. This is not entirely impossible, since the authors were able to get some of the discovered optimizations merged into LLVM. Further there were likely more improvements done between LLVM 18 to 20. The high variance between the single benchmarks could either stem from Minotaur optimizations or be caused by the system executing the benchmark. Nevertheless the overall results did not change significantly

³<https://chromium.googlesource.com/libyuv/libyuv/+8a93618f98e3bcd682615a704dd3410d0fee09e4>

between the benchmark runs. Another interesting observation is that about half of the benchmark programs were slower and half faster than the LLVM baseline. This seems to indicate an issue with the cost model used to estimate the found substitutions.

3.3 libyuv

In order to be able to even build libyuv and to not run into constant crashes in Alive2’s LLVM execution tool during the compilation phase, its third party dependencies were compiled without Minotaur. This would probably not have influenced the results to much since those dependencies seemed to be used mainly for auxiliary functionality. Due to us not compiling the third party dependencies comparing compile times makes little sense because the authors seemed to compile all sources using Minotaur, judging by their compile time. We were not able to run the benchmark suite of libyuv, which doubles as its test suite, since it was failing the tests when using Minotaur. Similar to the results for libgmp the regular LLVM build passed the tests with no issues.

3.4 Small LLVM IR Samples

These results from libgmp and libyuv prompted us to investigate further. We decided to try to reproduce some of the optimizations for short LLVM IR code snippets listed in the paper. Since the original C source code is not provided, we attempted to optimize the IR directly. However, we observed through different experiments that starting from LLVM IR rather than from the C code often leads to different optimization results, which could partially explain the poor results obtained.

In 5 of the 9 snippets tested, Minotaur failed to produce any optimization. Only in the case of example 3 Minotaur successfully generated optimized code that even then differed slightly from the one presented in the paper. In the case of example 6, the standard LLVM was able to produce the expected optimization but Minotaur failed to do so. Finally, in examples 7 and 8 Minotaur produced new code which featured more instructions than the original, non-optimized code. In example 7, however, standard LLVM also produced an optimization but it was different from the one presented in the paper.

We also attempted to reproduce optimizations reported by the paper authors in LLVM’s GitHub issues. In most cases tested^{4 5 6 7 8} both Minotaur and LLVM were able to produce the expected optimizations. However, in one case⁹ Minotaur

failed to reproduce the expected optimization. Only in one of the tested issues¹⁰ Minotaur produced an optimization that LLVM could not by eliminating an entire function.

In conclusion it can be said that we were not able to reproduce the results of the paper. The builds took significantly longer than expected. We also found issues with the correctness of the applied optimizations. Our runtime benchmark of libgmp did not show an improvement to the average runtime. To reproduce the results from the paper one would have to fix the caching, the correctness issues and like also adjust the parameters to allow for a longer more thorough search for optimizations.

4 Research Ideas

4.1 Improved caching

As previously discussed, caching, which should be a key mechanism for reducing compile time, does not appear to operate very effectively in the current version of Minotaur. A potential reason is that cached rewrites are keyed on the literal structure of extracted cuts, causing near misses on semantically equivalent cuts. A possible approach to this is the introduction of a canonicalization layer so that identical computations map to the same key. This could be implemented through the normalization of SSA identifiers, sorting of operands in commutative instructions and the standardization of other elements like vector masks. The impact of these changes could be evaluated by measuring improvements in cache hit rates, reduction of solver timeouts and compilation times, as well as by checking whether more rewrites are successfully applied across different programs.

4.2 Dynamic Cutting Depth using Profile Guided Optimization

One major down side of Minotaur at the moment are its long compile times. Even for small projects, which usually compile within seconds, it can take thousands of times longer, making it hours or even days. Improving on compile time could enable a more practical use of Minotaur. The main challenge is how to save on time while compiling and to not lose the benefits of Minotaur i.e. runtime performance improvements.

To achieve this one could try, as the paper suggested, to dynamically set the cutting depth. Determining the cutting depth for each cut is hard since one would have to know how critical a piece of code is to the overall runtime performance. One simple way of doing this would be to first build the given code without Minotaur and then benchmark it while gathering data with a profiler. This profiling data would then be used in a second compilation of the code, this time also using Minotaur, to determine the cutting depth dynamically.

⁴<https://github.com/llvm/llvm-project/issues/88168>

⁵<https://github.com/llvm/llvm-project/issues/86079>

⁶<https://github.com/llvm/llvm-project/issues/86068>

⁷<https://github.com/llvm/llvm-project/issues/86063>

⁸<https://github.com/llvm/llvm-project/issues/86061>

⁹<https://github.com/llvm/llvm-project/issues/88030>

¹⁰<https://github.com/llvm/llvm-project/issues/118114>

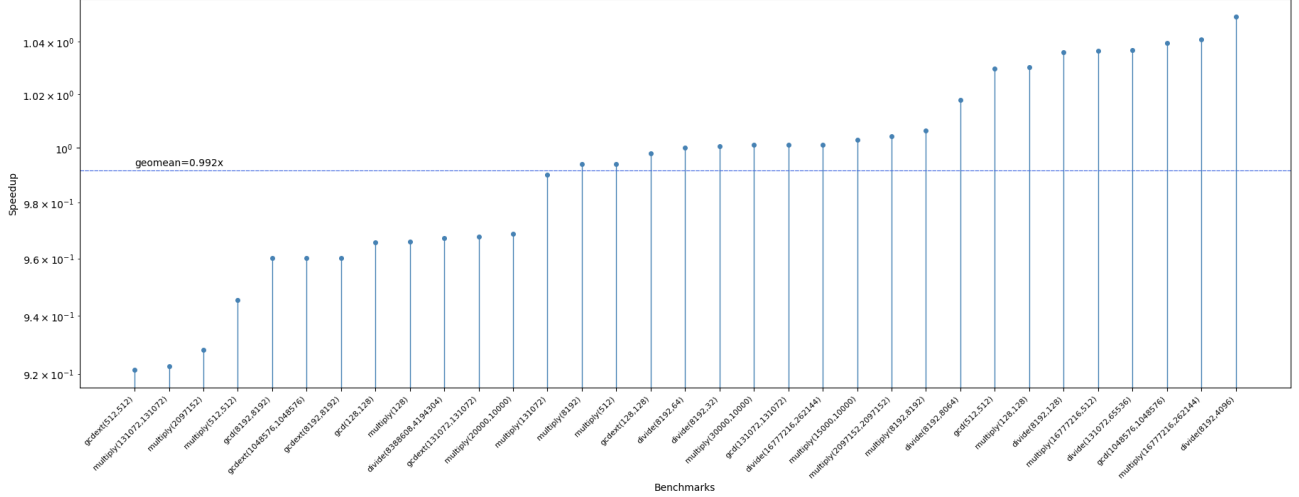


Figure 1: Our libgmp benchmark results

In theory two benefits could be derived from this approach, namely reducing total compile time, by not spending time on unimportant code like code that is for instance only ever executed once, and second, it could also enable Minotaur to spend more time on important code which actually impacts performance. Changes in compile time between the papers version of Minotaur and the augmented version would be the first important metric. But this is obviously not enough, one also have to measure the runtime of the produced code in order to determine if this idea delivers any benefits. Ideally the suggestion would improve compile times by not optimizing non critical code while keeping constant or maybe improving runtime of a program.

4.3 Arm Support

Currently Minotaur only supports x86 architectures. While it contains a large amount of x86, intrinsics it still uses LLVM IR which is platform-agnostic. When considering ARM, LLVM support for NEON is more robust and more similar to x86 used in Minotaur. Adding ARM support would entail implementing ca. 100 intrinsics (the x86 version of Minotaur uses 165), of which many are type duplicates, i.e. 32 bit and 64 bit variants. While this seems feasible, it would also require modifying the register allocator and verifying the formal verification in Alive2 to support ARM specific instructions. Furthermore, optimizations found on x86 might not be applicable to ARM and vice versa, so a new cost model would be needed. As a result this seems to be a significant engineering effort, which also requires advanced ARM benchmarking to determine which operations are faster on ARM hardware. Benchmarks similar to the ones used in the paper could be used to evaluate the performance of Minotaur on ARM hardware. Adding ARM support would increase the applicability of Minotaur and allow it to work on more hardware platforms.

4.4 Developer Tools

Currently Minotaur is largely a black box. While it is possible to inspect the transformations via cache-dump of the redis database, this is no very user-friendly format. An improvement would be better introspection tools to enhance Minotaur as a research tool specifically. The cache can be made queryable and interactive via a graphical interface. Instead of raw LLVM IR one could display the rewrites used in the synthesis. This goes hand in hand with statistics gathering about which cuts produce the most optimizations, which optimizations are most common and which optimizations produce the highest speedup in the benchmarks. For example Graphviz could be used to visualize the cuts and their optimizations and a snippet view could show code before and after optimizations. One can let a third party developer use Minotaur without the tools and with the tools and gather feedback. This would make Minotaur and its optimization more accessible to researchers and make it less cumbersome to use.

References

- [1] Zhengyang Liu, Stefan Mada, and John Regehr. Minotaur: A SIMD-oriented synthesizing superoptimizer. *Proceedings of the ACM on Programming Languages (OOPSLA2)*, 8(OOPSLA2):1–25, October 2024. doi: 10.1145/3689766. <https://doi.org/10.1145/3689766>.
- [2] Raimondas Sasnauskas, Yang Chen, Peter Collingbourne, Jeroen Ketema, Jubi Taneja, and John Regehr. Souper: A synthesizing superoptimizer. *CoRR*, abs/1711.04422, 2017. URL <http://arxiv.org/abs/1711.04422>.
- [3] Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. *CoRR*, abs/1211.0557, 2012. URL <http://arxiv.org/abs/1211.0557>.